

Git / Github

1. What is Version Control?
2. Need of Version Control.
3. principle of Version Control.
4. evaluation of Version Control.
5. Emergency of Git.
6. understanding Git.
7. Need Of Git Hub.
8. GIT deep dive.
9. GIT hands-on.

Git → 2005

CSS Cascading Styling Sheet.

IDE → Integrated development environment:-

Level ①: CSS Basic

Level ②: Color System and Background.

Level ③: Text Properties

Level ④: Box model CSS

Level ⑤: Display & position

Level ⑥: Flexbox, Grid & media Queries

Level ⑦: Animation, Transition & Transform

Project: my nba Clone.

1. Basic Syntax:-

Selector

h1 { font-size : 30px ;

↓
properties

↓
value

}

→ Declaration.

→ Selector:- The HTML element that you want to style.

→ Property: The attribute you want to change (like font, color, etc)

→ value: The Specific style you want to apply to the property (like red, bold, etc).

Lorem → to create random

Ex: <!DOCTYPE html>

<html>

<head>

<title> Color property </title>

<style>

h1 {

color: green

}

p {

color: red

}

</style>

</head>

<body>

<h1> This is the heading </h1>

<p> i am priyanshu </p>

</body>

</html>

* Inline Styling :->

- Direct Application : Apply styles directly to HTML elements using the style attribute.
- One-off changes : Ideal for single, unique style alteration.
- Can be cluttered : may lead to cluttered HTML if used extensively.
- Limited Reusability : Reduce the reusability of CSS rules in larger project.

ex: `<h1 style="color: green">this is the heading </h1>`

3. Including styling (External styling)

- separate CSS file : stores styles in separate.css

8. Selections (Descendant Selector)

`<title> Descendant selector </title>`

`<style>`

`div p { color: red }`

`p { color: blue }`

`</style>`

`<head>`

`<body>`

`<div>`

`<p>`

`lorem ipsum dolor sit amet`

`</p>`

`</div>`

`<p>`

`lorem ipsum dolor sit amet`

`</p>`

`</body>`

Q1 → generative pre-trained transformer

- Nested Targeting : Styles elements nested within a specified element.
- Syntax: separate selectors with spaces.
- Hierarchy-based: Allows styling based on the hierarchical structure of H.
- Specific styling: facilitates more targeted & specific styling of elements.

* Practice Set Level 1 *

- Create a heading and set the text color 'red'.
- Create a div with id # heading, include css using all 3 ways inline, style tag and external and observe property.
- Add comments to your css class
- ~~class~~ create a div, paragraph and heading & use id selector, element selector & class selector for them.
- Create two divs with id first and 2nd and define color for both using group selector.

* <!-- comment -->
or /* comment */

```
* <!doctype>
  <html>
    <head>
      <title> practice set 1 </title>
      <style>
        p {
          color: green;
        }
```


Level-2

Color system & background

9. Background-color property
10. Color system
11. Absolute units
12. Height & width property
13. Background-image property
14. Visibility property

⑨ Background color:-

Syntax: $\rightarrow$ background-color : color ;

⑩ Color-system $\rightarrow$

(color theory) $\rightarrow$

- RGB model : creates colors by mixing Red , Green , blue light sources.
- Additive model :- more light means increased brightness.
- Primary colors :- R, G, & B.
- Color depth $\rightarrow$

Ex: $\langle$ div style = "background-color :
rgb (255, 0, 0); " $\rangle$ first $\langle$ /div $\rangle$

```
<!doctype>
```

```
<html>
```

```
<head>
```

```
<title> Background-color </title>
```

```
<style>
```

```
#first {
```

```
background-color: orange;
}
```

```
#second {
```

```
background-color: white;
}
```

```
#third {
```

```
background-color: green;
}
```

```
</style>
```

```
</head>
```

```
<body> <b> <center>
```

```
<div id="first"> first </div>
```

```
<div id="second"> Second </div>
```

```
<div id="third"> Third </div> </b> </center>
```

```
</center> <div style="background-color: rgb(0, 0, 255);">
```

```
<center> forth </center> </div>
```

```
<body>
```

```
</html>
```

* * * * * Hex Color model * * * * *

- Hexadecimal codes $\Rightarrow$ Represents colors using hexadecimal values, consisting of 6 digits combined from numbers & letters (A-F).

- Syntax $\Rightarrow$ Written as #RRGGBB.

- Easy color matching:- facilitates easy color matching with graphic design tools and branding colors.
- Web Standards: widely supported and a common standard for defining colors in web design.

Ex:->

```
<div style = "background-color: #ffcd2">
first</div>
```

*** Absolute units ***

- Definition: Pixels (px), are fixed-size units, presenting a dot on a computer screen.
- Precision: Allows for precise control over element dimension.
- Graphics & web design:-
Commonly used in graphics & web design for setting font sizes, margins, and more.
- Cross-Browser Consistency: provides consistency across different browsers.
- High-DPI Displays: can vary in appearance on high-DPI (dots per inch) displays.

*** Height & width property ***

```
<head>
```

```
<title> Height </title>
```

```
<style>
```

```
. box { height: 40px; width: 40px; }
```

```
# box1 { background-color: red; }
```

```
# box2 { background-color: blue; }
```

```
# box3 { background-color: green; }
```

</style>

<head>

<body>

<div id="box1" class="box">Box 1 </div>

<div id="box2" class="box">Box 2 </div>

<div id="box3" class="box">Box 3 </div>

</body>

*** Background Image Property ***

- usage: Adds an image as a background to elements.

- syntax: Define using background-image:

url('path/to/image');

- Repetition: Control image repetition using background-size.

- Background-size.

- Background-Attachment: sets whether the background image scrolls with the element or remains fixed.

- shorthand (color, image, repeat, attachment, position).

W-3 [L-23] Podded engine:

Why put engines in pods on wing?

- better access to work on engines

- Passenger safety during accidents.

Ex: <title> Background image </title>

<style>

#icon {

color: red;

background-color: red;

height: 50px;

width: 50px;

background-image: url(mynta.png);

}

</style>

</head>

<body>

<div id="icon"></div>

</body></html>

Ex: <!DOCTYPE html>
<head>

<title> visibility </title>

<style>

box1 {

height: 50px;

width: 50px;

}

</style>

</head>

<body>

<div id="box1" class="box"> box1 </div>

<div id="box2" class="box"> box2 </div>

</body>

Ex:->

box2 {

height: 50px;

width: 50px;

visibility: hidden;

}

Ex: <!doctype html>
<html lang = "en">

< head >

< title > Practice set 2 </title>

< style >

}

bar {

background-color: aquamarine;

color: rgb(103, 18, 18, 0.8);

}

product {

background-image: url(clean.webp);

width: 800px;

height: 500px;

background-position: center;

background-size: 400px;

main { height: 500px;

background-color: blanchedalmond; }

</style>

</head>

< body >

< header >

< div id = "bar" > this is my new bar </div>

</ header >

< main >

< div id = "product" > this is my new product </div>

</ main >

< footer >

</ footer >

</ body > </ html >

Level: 3 (Text Properties)

- Text-Align Property
- Text-Decoration property
- Text-transform property
- line-height property
- font properties
- font family
- Icon using fonts

Ex: 1

1

text-align

```
<head>
```

```
<title> Text Align </title>
```

```
<style>
```

```
  .box { height: 100px; width: 100px; }
```

```
  #box1 { background-color: red; text-align: center; }
```

```
  #box2 { background-color: green; text-align: left; }
```

```
  #box3 { background-color: blue; text-align: right; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h3 id="box1" class="box">Box 1 </h3>
```

```
<h3 id="box2" class="box">Box 2 </h3>
```

```
<h3 id="box3" class="box">Box 3 </h3>
```

```
</body>
```

Syntax: text-align = center/left/right

2

Text decoration: -

```
#box1 {
```

```
  [Syntax: text-decoration: underline;
```

```
  " " : overline;
```

```
  " " : line-through;
```

```
}
```

• → class definition
→ id definition

** Syntax: → # box1 { text-decoration-style : dashed;
" " " : double;
" " " : solid;
" " " : wavy;
}

*** syntax: → # box1 { text-decoration-color : red; }

17: Text-Transform Property



box1 {

text-transform : uppercase

" " : lowercase;

" " : Capitalize;

" " : none; }

18. line height

<!doctype html>

<html lang="en">

<head>

<title> line Height </title>

<style>

article {

line-height : 1.5px ;

}

</style>

</head>

<body>

<p class="article"> my name is priyanshu Raj daga.

</body>

19: font Property

first1 { font-size : 5px; }

second2 { font-weight : 600; }

third { font-style: normal / oblique / italic }

20: font family

- usage: Defines which font should be used for text within an element.
- Specific fonts: Common choice include Arial, Segoe UI, Times New Roman, and others.
- fall back mechanism: incorporate a fallback font family in case the primary font is unavailable; helps in maintaining the site aesthetics.
- web safe fonts: employ web-safe fonts to ensure consistency across different browsers & operating systems.
- generic family: Always end the font family list with a generic family like serif or sans-serif as a last resort option.

21: Icons using fonts

```
<head>
<title>Icons</title>
<script src = "https://kit.fontawesome.com/
43290fa92d.js" cross origin = "anonymous">
</script>
</head>
<body>
<i class = "fa-solid fa-house"></i>
<i class = "fa-brands fa-facebook"></i>
<i class = "fa-brands fa-tiktok"></i>
<i class = "fa-solid fa-poo" style = "font-size:
40px;"></i>
</body>
```

Practice Set Level: 3

- Create an heading at the centre & make capitalize.
- use font family for the whole page to Times New Roman.
- create one div inside another div.

set id and text outer to outer div.

set id and text inner to inner div.

set id & text outer to inner div.

text size 25px.

set inner to 10px.

set inner to 10px.

- use icons from fontawesome.com & use icons of LinkedIn and GitHub.

Level-4 : (Box model):

22.* What is Box model: →

- Core concept:

central concept in CSS that outlines the design and layout of elements on the web page.

- Components: Consists of four main components - margin, border, padding, and content.
- margin: The space outside the border, separating the element from others.
- Border: The outline that encapsulates the padding and content.
- padding: The space b/w the border & the actual content providing a buffer.
- Content: The innermost layer where text, images, or other media are housed.

```

ex-: <head>
      <title> padding </title>
      <style>
        * {
          margin: 5px 10px 15px 20px;
          background-color: yellow;
        }
      </style>
    </head>
    <body>
      <button id="button1">Click me</button>
    </body>
  
```

* What is Box-model:

- * padding property
- * margin property
- * border property

```

ex.: <!doctype html>
      <html lang="en">
      <head>
        <title> Color and visibility </title>
        <style>
          * {
            margin: 0;
            padding: 0;
          }
        
```

• Pad &

```

height: 100px;
width: 100px;
padding: 20px;
box-sizing: border-box;
}

```


Practice det level → 4

Level → 05 (Display & Position)

- * Display property
- * Responsive website
- * Relative units
- * Position property
- * Semantic Tags.

Block-Elements

- New line : Start on a new line.
- Full width : Take up all horizontal space
- Styling : can have margins & padding
- Size : width & height can be set
- ex : <div>, <p>, <h1>, ,

Inline-Elements

- Flow
- Width
- No break
- Limited styling
- ex :

① Display property :-

• box {

height : 100px;

width : 100px;

background-color : blueviolet;

margin : 10px;

text-align : center;

border : 5px solid black;

display : block;

display : block :-

box

box

box

↓
set size

display : inline

box box box

↓

not set size

display : inline-block

box box box

↓

set size

display : none

→ not fix size

② Responsive design →

- Adapts layout for different screen sizes
- flexible layouts
- optimizes images & assets
- enhances user experience on mobile & desktop

③ Relative unit (for responsive design):-

ex: #first {

height: 200px;

width: 200px;

bg-color: aqua;

font-size: 25px;

}

#second {

bg-color: blueviolet;

width: 50%;

height: 30%;

}

* Relative units (VW/VH):-

<head>

<title> unit VH & VW </title>

<style>

#first {

height: 50vh;

width: 90vw;

bg-color: red;

}

</style>

</head>

<body>

<div id="first"></div>

</body>

29 (Position Property):

Static Relative absolute Fixed

- Static $\Rightarrow$ Elements follow the normal document flow.

(top, right, bottom, left,

z-index would not work).

- Relative $\Rightarrow$ Element's position adjusted from its normal position.

- Absolute: Positions element relative to the nearest positioned ancestor.

- Fixed $\Rightarrow$ Element positioned relative to the viewport, does not move on scroll.

- Position property (z-index)

- container {

position: relative;

}

- box1, box2 {

position: absolute;

border: 3px solid black;

width: 100px;

height: 100px;

text-align: center;

font-size: 25px;

}

- box1 {

background-color: red;

left: 20px;

top: 60px;

z-index: 2;

}

- box2 {

background-color: aqua;

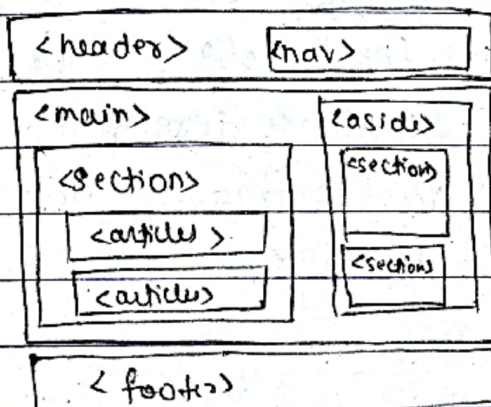
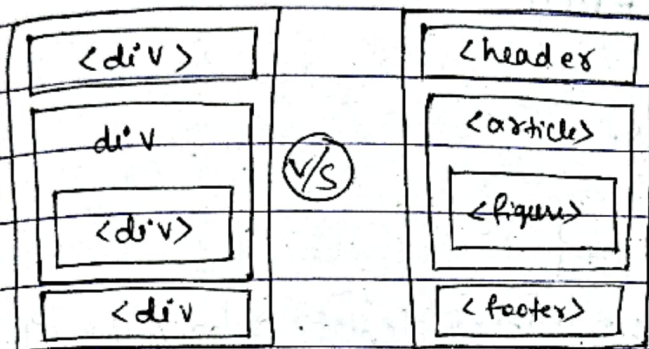
left: 60px;

top: 20px;

z-index: 1;

- stacking order: determines the stacking order of elements along the z-axis.

* * * * * Semantic Tags * * * * *



Semantic Tag

- meaningful: describe content
- SEO: good for search engines
- Accessibility: useful for screen readers

Ex: <header>, <footer>,
<article>, <section>, <nav>

Non-Semantic Tags:→

- Generic: No specific meaning
- for styling: used for layout
- No SEO: Not SEO-friendly
Ex: <div>, <div>...

Practice Set level 5

Level-6

31. float property
32. What is flexbox?
33. flex model
34. flexbox direction.
35. properties: flexbox container.
36. properties: flex items.
37. Grid layout
38. media Queries.

31. float property

- Element Alignment: → Allows elements to be aligned to the left or right within their containing element.
- Values: → Can take values like 'left', 'right', or 'none' to determine the floating direction.
- Old layout Technique: → less commonly used with the advent of flexbox: -

ex: . container {

height: 110px;

width: 300px;

border: 1px solid

}

• box {

width: 100px;

height: 100px;

margin: 5px;

}

• box1 {

background-color: red;

float: right;

}

• box2 {

background-color: blue;

float: left;

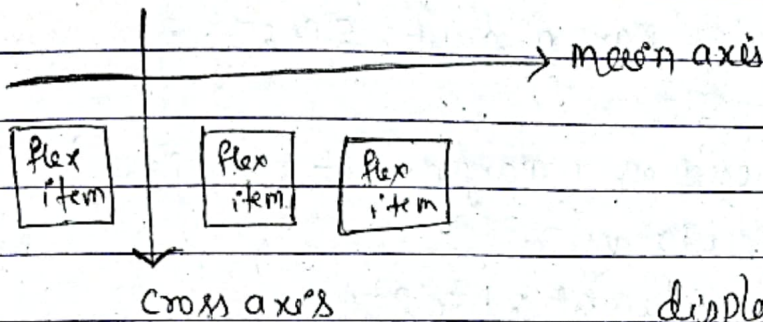
}

(32) flexbox ?

→ flexbox is a one-dimensional layout method for arranging items in rows or columns.

items flex (expand) to fill additional space or shrink to fit into smaller space.

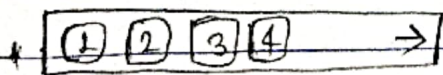
(33) * flex model:→



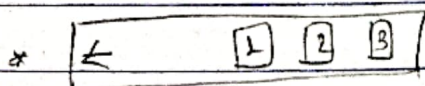
display: flex;

(34) Flexbox Direction:→

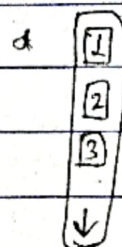
- Property Name:→ flex-direction is the property used to define the direction in a flex container.
- Row layout:- Row value aligns the flex items horizontally in a left-to-right fashion.
- Column layout: Column value stacks the flex items vertically from top to bottom.
- Reverse Direction: Adding -reverse to flex row or column (as in row-reverse or column-reverse) reverse the order of the items.



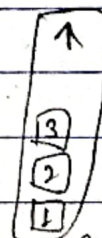
flex-direction: row



flex-direction: row-reverse



flex-direction: column



flex-direction: column-reverse

column-reverse

px: . {

margin: 0;

padding: 0;

}

• box {

height: 100px;

width: 100px;

border: 3px solid black;

margin-right: 5px;

}

heading { margin-left: 200px; }

container {

height: 150px;

width: 600px;

padding: 10px;

margin: 20px;

border: 3px solid black;

display: flex;

flex-direction: row-reverse;

}

box1 { background-color: aqua; }

box2 {

}

box3 {

}

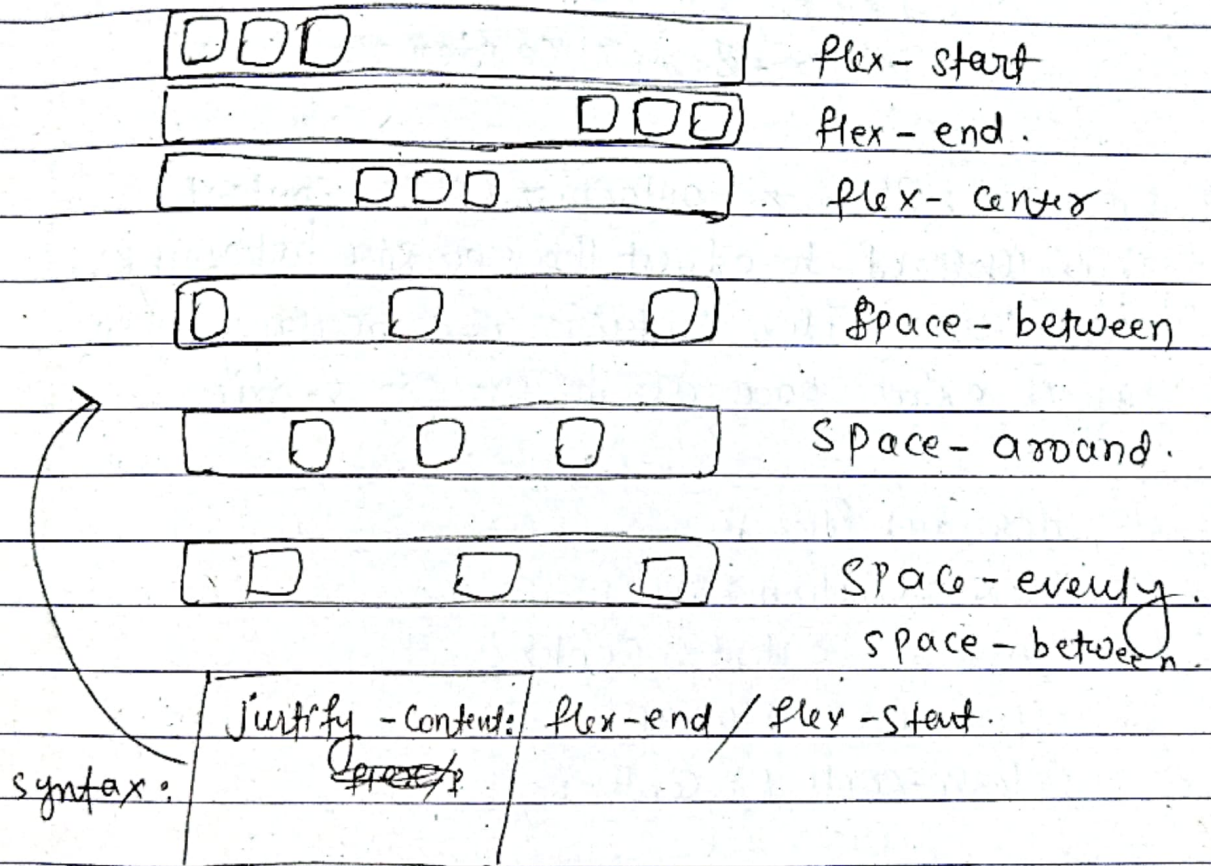
box4 {

}

display: flex;

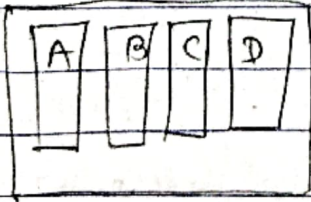
flex-direction: row/row-reverse
or column/column-reverse.

- ①
- ③ properties : flexbox Container (justify Content) :->
- Alignment : Aligns flex items along the main axis.
 - flex-start : items align to the start of the flex.
 - flex-end : items align to the end of the flex container.
 - center : items are centered within the flex container.
 - space-between / space-around / space-evenly : Distributes space between items evenly.

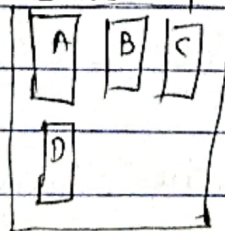


② properties : flexbox Container (flex-wrap) :-

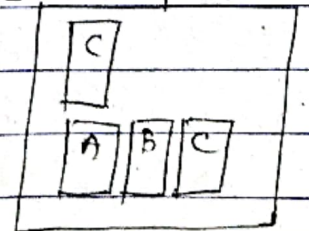
① nowrap (default)



② wrap



③ wrap-reverse



* Properties: flexbox Container (Align items) :-

This property is used to align the flex container's items along the cross-axis, which is perpendicular to the main axis.

```
Ex: display: flex;
flex-direction: row;
justify-content: center;
align-items: center;
```

* Properties: flexbox Container (Align Content) :-

It is utilized to adjust the spacing between flex lines within a flex container, particularly when there is extra space along the cross-axis.

```
display: flex;
flex-direction: row;
justify-content: center;
flex-wrap: wrap;
align-content: center;
```

* ③ Properties: flex items (flex shrink) :-

```
#box 3 {
```

```
background-color: yellow;
flex-shrink: 4;
```

```
}
```

→ The "flex-shrink" property in CSS determines how much a flex item will shrink relative to other items in the flex container if there is insufficient space.

36. Properties: flex item (flex grow):

```
# box3 {
```

```
background-color : yellow;
```

```
flex-grow : 1;
```

```
}
```

→ The "flex-grow" property in CSS specifies how much a flex item will grow relative items in the flex container when additional space is available.

36. Properties: flex items (order)

```
ex: # box1 {
```

```
background-color : aqua;
```

```
order : 3
```

```
}
```

```
# box2 {
```

```
background-color : blueviolet;
```

```
order : 1;
```

```
}
```

```
# box3 {
```

```
background-color : yellow;
```

```
order : 4;
```

```
}
```

```
# box4 {
```

```
background-color : tomato;
```

```
order : 2;
```

```
}
```

→ The "order" property in CSS allows you to define the sequence in which flex items appear within the flex container, overriding their original order in the HTML.

37. Grid

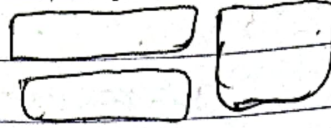
flexbox :→

one-dimensional layout



grid :→

multi-dimensional layout



- 2D layout system for rows & columns.
- Activate with display: grid;
- Children become grid items.
- Define structure with grid-template properties.
- Individual units called grid cells.

ex-:

• Container {

display: grid;

grid-template-columns: 50px 50px;

grid-template-rows: 50px 50px;

}

• item 1 {

grid-column: 1/2;

grid-row: 1/2;

background-color: lightblue;

}

• item 2 {

grid-column: 2/2;

grid-row: 1/2;

background-color: lightgreen;

}

• item 3 {

grid-column: 1/2;

grid-row: 2/2;

background-color: lightpink;

}

• item 4 {

grid-column: 2/2;

grid-row: 2/2;

background-color: lightyellow;

38. (Media Queries)

- Tailor styles for specific device characteristics.
- Use to create responsive web designs.
- Apply styles based on conditions like screen size.
- Syntax: `@media (condition) {css rules}`.
- Can combine multiple

ex: `@media screen and (max-width : 768px) {`
 `.container {`
 // write style here
 }
}

declaration mediaType

↑ ↑

styles to apply when all conditions are met specifying amount of screen to cover.

ex-: `.box {`
 `height : 100px;`
 `width : 100px;`
 `background-color : lightsalmon;`
 `}`

 `@media screen and (width : 250px) {`
 `border : 5px solid red;`
 `}`

38. Media Queries (min-width)

ex: `.box {`
 `height : 100px;`
 `width : 100px;`
 `background-color : lightsalmon;`
 `}`

 `@media screen and (min-width : 300px) {`
 `.box {`
 `height : 150px;`
 `width : 150px;`
 `}`
 `}`

(max-width : 300px)

Level-7 (Animation, Transition & Transfer)

- 39. Pseudo classes
- 40. Transitions
- 41. CSS Transform
- 42. Animation.

* Pseudo classes :->

- Used to define special states of HTML elements.
- Syntax: Selector : pseudo-class { styles }
- Common examples: hover, active, first-child.
Target elements based on their position or user action.

```
Ex:- .btn {  
    height : 20px;  
    width : 70px;  
    border : 1px solid blue;  
    border-radius : 5px;  
    background-color : aliceblue;  
}
```

```
.btn :hover {  
    height : 25px;  
    width : 80px;  
    border : 1px solid red;  
}
```

```
.btn :active {  
    height : 25px;  
    width : 80px;  
    border : 1px solid red;  
    background-color : indianred;  
}
```

* 40. Transitions: →

CSS transition is a property that enables smooth animation b/w changes in CSS property values.

- **Transition-property**: Defines which CSS properties will transition changes in CSS property values.
- **Transition-duration**: Sets how long the transition lasts.
- **Transition-timing-function**: Controls the speed curve of the transition.
- **Transition-delay**: Specifies a delay before the transition starts.

Ex: • btn {

height: 20px;

width: 70px;

border: 1px solid blue;

border-radius: 5px;

background-color: lightblue;

transition-property: all;

transition-duration: 1s;

transition-timing-function: ease-in-out;

transition-delay: 1s;

}

• btn: hover {

height: 25px;

width: 80px;

border: 1px solid red;

}

• btn: active {

height: 25px;

width: 80px;

border: 1px solid red;

background-color: indianred;

}

* 4.1 (CSS Transform):-

- Allows modification of an element's shape and position.
- Can perform operations like rotate, scale, & translate.
- Does not affect the layout of surrounding elements.
- Used to create visual effect.
- rotate()
- Scale()
- Skew() & translate.

Ex:-

```
• box {  
    height : 50px ;  
    width : 50px ;  
    padding : 5px ;  
    margin : 20px ;  
    border : 1px solid black ;  
    border-radius : 5px ;  
    transition-property : all ;  
    transition-duration : 1s ;  
    transition-timing-function : ease-in-out ;  
}
```

```
# box 1 { background-color : lime ; }
```

```
# box 1 : hover { transform : rotate (45deg) ; }
```

```
# box 2 { background-color : blueviolet ; }
```

```
# box 2 : hover { transform : rotate (180deg) ; }
```

* Scale Property

```
# box 1 { background-color : lime ; }
```

```
# box 1 : hover { transform : scale (2) ; }
```

* Translate :-

```
# box1 { background-color : lime; }
```

```
# box1 : hover { transform : translate (50px); }
```

* Skew :-

```
# box1 { bgcolor : lime; }
```

```
# box1 : hover { transform : skew (45deg); }
```

* * 42 (Animation) :->

- Animation-name : Specifies the name of the @keyframes defined animation.
- animation-duration : Defines the total time the animation takes to complete one cycle.
- animation-timing-function : Controls the pacing of the animation (e.g. linear, ease-in).
- animation-delay : Sets a delay before the animation starts, allowing for a pause before initiation.
- Animation-iteration-count : Indicates the no. of times the animation should repeat.
- Animation-direction : specifies the dirⁿ of the animation, allowing for reverse or alternate cycles.

Ex:-> .box {

height = 10px;

width = 10px;

border = 2px solid black;

border-radius : 5px;

bgcolor : plum;

position : relative;

top : 50px;

left : 50px;

animation-name: ghunakkad;

animation-duration: 2sec;

Animation-timing-function: ease-in-out;

animation-delay: 1sec;

animation-iteration-count: 4;

animation-direction: alternate;

or, $\downarrow$ animation: ghunakkad 2s ease-in-out
1sec infinite alternate;
}

@keyframes ghunakkad {

0% from { left: 10px }

50% to { left: 200px }

60% left { 160px }

100% { left: 200px }

}



Project

(Mynta clone)